DEEP-LEARNING WORKFLOW

Plant Disease Detection Using CNN

An end-to-end deep-learning workflow for classifying plant-leaf images.



Made with GAMMA

THE PROBLEM

Manual inspection is slow and inconsistent

Leaf diseases create visible symptoms, but reviewing images manually does not scale well.

This project explores whether a CNN can learn visual differences from labeled leaf images.

Output: one predicted class with a probability for each known category.

Scope: image-level classification, not field diagnosis, segmentation, or bounding boxes.



A leaf image becomes one class predictio

Made with GAMMA

PROJECT SHAPE

The project turns labeled images into a reusable classifier

01

Prepare

Prepare a labeled image dataset.

02

Build

Build training and validation datasets automatically.

03

Train

Train a custom CNN with augmentation and callbacks.

04

Evaluate

Evaluate with validation metrics and a classification report.

05

Export

Export the model and class-name mapping for inference.

Made with GAMMA

DATASET CONTRACT

The dataset is organized around class folders

Source: Kaggle dataset vipoooool/new-plant-diseases-dataset

Expected structure: train, valid, and optional test folders with one subfolder per class.

Class discovery: non-empty training subfolders are detected and sorted alphabetically.

Image support: JPG, JPEG, PNG, BMP, and WEBP.

Important: exact class and image counts are printed at runtime, not assumed in advance.



Folders define the labels the pipeline c

Made with GAMMA

AUTOMATED PIPELINE

The workflow is automated from download to export

Each stage prepares the next one.



Made with GAMMA

INPUT PREPARATION

Augmentation helps the model see realistic variation

STANDARDIZE

160 × 160

Images are resized to 160 × 160 pixels and grouped in batches of 32.

VARY

Flip · rotate · zoom

Training transforms include horizontal flips, small rotations, zoom, and contrast changes.

SCALE

0–1 pixels

A Rescaling layer converts pixel values from 0–255 to approximately 0–1.

Fallback validation

If no validation folder exists, the notebook creates an 80/20 training-validation split.

Efficient input

TensorFlow prefetching overlaps input preparation with model execution.

Made with GAMMA

MODEL ARCHITECTURE

The CNN builds features from simple patterns to rich representatio ns

Early layers detect local patterns; later layers combine them into class evidence.



The final output contains one probability for each known category.

Made with GAMMA

TRAINING CONTROL

Training is designed to preserve the best validation state

OPTIMIZER

Adam

Learning rate 0.001.

LOSS

Sparse categorical cross-entropy for integer labels.

INITIAL SCHEDULE

Up to five epochs

Checkpoint: save the lowest validation loss.

Early stopping: stop when progress stalls.

Learning-rate reduction: reduce the rate on plateaus.

Five epochs is a short initial run and is not proof of convergence.

Made with GAMMA

EVALUATION

Evaluation must go beyond a single accuracy number

Actual values must come from a completed run. The supplied documentation contains no fixed measured result.

Training curves
Training and validation accuracy and loss plotted across epochs.

Best checkpoint
Evaluation on the validation dataset.

Classification report
Accuracy, loss, precision, recall, F1-score, and support.

Run-dependent evidence

Accuracy

Loss

F1-score

No values shown until a completed run supplies them.

Next checks: held-out test evaluation, confusion matrix, class-wise metrics, and misclassified examples.

Made with GAMMA

INFERENCE AND EXPORT

The saved model can be used for image inference

Model artifact: plant_disease_cnn.keras

Label artifact: plant_disease_classes.json

At inference, resize an RGB image to 160 × 160, predict probabilities, and select the highest-probability class.

Keep the JSON file with the model so output indices map to the correct class names.

Do not divide by 255 again because the saved model already includes Rescaling(1/255).



Input image: RGB resize, prediction, class mappi

Made with GAMMA